ACTIVIDAD 3: ENFOQUE MODULAR Y DISEÑO TOP-DOWN

Manipulación de Cadenas y de Punteros

Autores: Sofia Mojica, Gabriel Zacarías, Mateo Roberts y Leonel De Armas
Institución: Laboratorio de Programación (LPR), Escuela Técnica 5° 3° A-B
Docentes a cargo: Prof. York Mansilla y Prof. Yamil Ganduglia
Fecha: 15 de octubre de 2026

---

1. Introducción

El presente informe expone el análisis y desarrollo de la Actividad 3 del Laboratorio de Programación, centrada en la metodología de diseño Top-Down. Este enfoque descompone un problema macro en módulos lógicos reutilizables denominados funciones. El documento detalla los pilares de la programación estructurada mediante el diseño de una calculadora modular de área en C++, examinando la importancia de la validación sintáctica, la inmutabilidad de la memoria RAM y la contención de errores mediante filtros de entrada.

2. Desarrollo Técnico (Glosario e Ingeniería del Software)

- **Diseño Top-Down:** Estrategia analítica que subdivide un algoritmo complejo en subprocesos independientes de nivel inferior.
- **Modularizar:** Fragmentación del código fuente para aislar funciones operativas específicas, mejorando el mantenimiento de los sistemas.
- **Prototipo de Función:** Firma contractual que anticipa al compilador la interfaz (retorno, nombre y argumentos) de un módulo antes de su ejecución.
- **Retorno (Return):** Mecanismo de descarga que transfiere el resultado procesado por un subprograma de regreso al contexto de llamada.
- **Constante (`const`):** Directiva de compilación que fija un valor de forma persistente en la memoria de solo lectura durante el ciclo de vida del proceso.

3. Distribución de Roles y Tareas del Equipo

Integrante	Rol e Implementación Técnica
Sofía Mojica	Diseño e implementación del Filtro de Consistencia condicional para la validación geométrica del radio.
Gabriel Zacarías	Desarrollo de la Función Modular matemática en C++ para el cálculo del área e integración del prototipo.
Mateo Roberts	Estructuración del repositorio base `calculoarea`, renderizado de la diana ASCII y control de versiones en GitHub.
Leonel De Armas	Diseño del Diagrama de Flujo, resolución del cuestionario técnico y redacción del informe final en formato APA v7.

4. Estructura del Repositorio del Proyecto

t
/CALCULOAREA/
├── .gitignore <-- Archivo para que Git ignore carpetas (.vscode)
├── LICENSE <-- Archivo de licencia MIT del proyecto (Uso escolar)
├── README.md <-- Archivo explicativo del programa desarrollado
├── docs/
│ └── InformeProyectoCALCULOAREA.pdf <-- ENTREGABLE (Informe PDF en Normas APA v7)
├── src/
│ └── main.cpp <-- Código fuente modular en C++
└── capturas/
├── compilacion_area.png <-- Captura de la terminal compilando y corriendo el programa
└── diagrama_flujo.png <-- Captura o foto del diagrama de flujo del programa

5. Respuestas al Cuestionario Técnico de Control

Pregunta 1: El Prototipo de la Función

El compilador de C++ realiza un análisis sintáctico secuencial (de arriba hacia abajo). Si se elimina el prototipo de la línea 15, al procesar el bloque `main()` la terminal se topará con el identificador `calcularAreaCirculo` sin conocer sus requisitos de memoria previos. Al no saber cuántos bytes requiere el parámetro o qué tipo de dato retorna, el compilador interrumpe el proceso arrojando un error de tipo *"identifier not found"* o *"scope compile error"*. El prototipo actúa como una declaración anticipada necesaria para mapear la firma en la tabla de símbolos del compilador.

Pregunta 2: El Filtro de Consistencia

Desde el punto de vista geométrico y físico, un radio de 0 cm representa un punto dimensional nulo (sin superficie), y un radio de magnitud negativa (como -5 cm) constituye una incongruencia espacial inexistente en el espacio euclidiano. Permitir que el sistema ejecute la operación matemática arrojaría un área errónea o matemáticamente inválida. Controlar esto mediante un filtro condicional de consistencia preventivo es imperativo para blindar el software, evitando desperdiciar ciclos de reloj de la CPU en procesar datos de entrada corruptos.

Pregunta 3: Proteger Constantes en RAM

En sistemas críticos (aviación o medicina), una alteración involuntaria en una variable de control debido a desbordamientos de memoria (*buffer overflow*) o errores de asignación lógica podría provocar accidentes catastróficos, como alterar los cálculos de trayectoria de vuelo o dosificar erróneamente un fármaco. El uso de la palabra clave `const` instruye al compilador a colocar dicho valor en segmentos de memoria protegidos de "solo lectura". Si un bug del software intenta reescribir este espacio, el compilador bloquearía la construcción del ejecutable de inmediato, forzando la detección del fallo antes de poner el sistema en producción.

6. Conclusión

El desarrollo modular del software mediante funciones independientes mitiga la fragilidad del código centralizado. La aplicación de la técnica Top-Down permite delegar responsabilidades matemáticas precisas a bloques aislados en la memoria RAM,

facilitando una depuración focalizada. El correcto diseño de filtros y el bloqueo de constantes universales aseguran las bases de robustez algorítmica exigidas para el desarrollo de los proyectos técnicos de integración anual.

7. Enlaces y Aportes Individuales de Integrantes

1. Leonel De Armas:
https://github.com/tomasdearmas99-png/ACTIVIDAD-3-ENFOQUE-MODULAR-Y-DISE-O-TOP-DOWN---LPR-5-3-A-B.git